DOCUMENTACIÓN DE SIMULADOR: ASIGNACIÓN DE MEMORIA

- **Nombre del Estudiante:** Jeackeline Jimena Menendez Arreaga
- **Curso:** Sistemas Operativos
- **Herramienta utilizada:** Visual Studio Community (C#, Windows Forms)

1. Descripción de la Solución y Arquitectura

Se desarrolló una aplicación interactiva en C# con Windows Forms que simula la asignación y gestión de memoria principal. El sistema implementa las siguientes características técnicas clave:

- **Definición de Bloques y Segmentos:** Permite al usuario inicializar la memoria especificando un tamaño total y una serie de bloques personalizados (segmentos de diferentes tamaños separados por comas o espacios). Si se deja en blanco, toma por defecto un bloque contiguo con el total de la memoria.

- **Estrategias de Ajuste Dinámicas:** Incorpora un selector con tres algoritmos de asignación: **Primer Ajuste (First Fit)**, **Mejor Ajuste (Best Fit)** y **Peor Ajuste (Worst Fit)**. Si el usuario cambia de estrategia en el momento de ejecución, el sistema reasigna automáticamente los procesos activos para comparar el comportamiento de los algoritmos.

- **Gestión de Fragmentación y Estado:** Calcula y muestra en tiempo real métricas clave como el uso de memoria, la fragmentación interna, el estado de los bloques (Libre/Ocupado) y una representación gráfica interactiva.

- **Operaciones Avanzadas:** Incluye botones para asignar procesos, liberarlos (con actualización de historial), modificar tamaños de procesos activos en tiempo real y reiniciar completamente el simulador.

2. Caso de Prueba Definido (Secuencia de Programas)

Para cumplir con los requerimientos de la práctica, se ejecutó un escenario de prueba utilizando **3 programas que llegan a la memoria en un orden definido**:

- **Configuración Inicial de Memoria:**
    a. Memoria Total: `1000 KB`
    b. Bloques definidos: `300, 400, 300` (Tres bloques fijos de diferentes tamaños).
- **Estrategia seleccionada:** *Mejor Ajuste (Best Fit)*
- **Secuencia de llegada de los 3 programas:**
    c. **Programa P1:** `250 KB`
    d. **Programa P2:** `350 KB`
    e. **Programa P3:** `120 KB`

3. Resultados y Comportamiento Observado durante la Prueba

- **Paso 1 (Inicialización):** El sistema dibuja y estructura los tres bloques de memoria definidos (`300 KB`, `400 KB` y `300 KB`), marcándolos inicialmente en color verde como *Libres* con sus respectivas direcciones de inicio (0, 300 y 700).
- **Paso 2 (Llegada de P1 - 250 KB):** Bajo la estrategia de *Mejor Ajuste*, el sistema evalúa los bloques libres disponibles que tienen espacio suficiente (todos cumplen inicialmente). Selecciona el bloque de `300 KB` (el que deja un residuo menor de 50 KB frente al de 400 o 300). P1 ocupa el espacio y genera un cálculo de fragmentación interna de 50 KB.
- **Paso 3 (Llegada de P2 - 350 KB):** El algoritmo analiza los huecos restantes (`50 KB` libres en el primer bloque, `400 KB` en el segundo y `300 KB` en el tercero). El único bloque donde cabe el programa de 350 KB es el de `400 KB`. Se le asigna, dejando una pequeña fragmentación interna de 50 KB.
- **Paso 4 (Llegada de P3 - 120 KB):** Quedan disponibles el bloque de 300 KB al final (totalmente libre) y los residuos de los bloques anteriores. Aplicando *Mejor Ajuste*, P3 se asigna al bloque libre de `300 KB` (ya que es el que mejor se ajusta en tamaño comparado con repartir en huecos más grandes), registrando su estado como *Asignado* en el panel derecho de procesos y actualizando la gráfica visualmente.
- **Liberación y Modificación:** Al liberar o modificar un proceso mediante la interfaz o haciendo clic directamente sobre los bloques gráficos, el sistema responde de forma inmediata manteniendo la integridad de las estructuras de datos y reflejando los cambios en las tablas.